

Contributing to BST

Welcome. This project grows by contribution — new theorems, new toys, new connections. Whether you're a human mathematician, a CI, or a team of both, here's how to add to the program.

The One Rule

Every claim needs a toy. No exceptions. If you can't test it computationally, it's speculation (put it in `notes/maybe/`). If the toy passes, it's a candidate for the theorem registry.

What We Accept

New Theorems

- Prove something new, or strengthen an existing result
- Classify its $AC(0)$ depth (how many layers of counting?)
- Connect it to the theorem graph (what does it depend on? what does it enable?)
- Build a toy that verifies it empirically

New Toys

- Computational experiments testing claims
- Must be self-contained Python scripts
- Must produce clear PASS/FAIL verdicts
- Must include a score line: Toy NNN -- SCORE: X/Y

Proof Improvements

- Tighten a bound
- Close a gap (CONDITIONAL $\rightarrow$ PROVED)
- Find a simpler proof (lower $AC(0)$ depth)
- Catch an error (RETRACTED is honest, not shameful)

New Derivations

- Derive a new physical constant from D_{IV}^5
- Connect BST to a new domain
- Extend the $AC(0)$ program to a new problem

Quality Gates

Every PR must pass these checks before merge:

Gate 1: Toy Evidence

- ☐ New theorem has an accompanying toy
- ☐ Toy runs cleanly: `python3 play/toy_NNN_description.py`
- ☐ Score is stated and honest (6/8 is fine — state what failed and why)

Gate 2: $AC(0)$ Classification

- ☐ $AC(0)$ depth is stated
- ☐ Depth is justified (what are the counting operations at each level?)
- ☐ If depth > 3 , explain why simpler tools don't work

Gate 3: Registry Entry

- ☐ Theorem is registered in `notes/BST_AC_Theorem_Registry.md`
- ☐ `T_id` is the next available (check the registry — no gaps, no duplicates)
- ☐ Status is accurate: PROVED / CONDITIONAL / EMPIRICAL / OPEN
- ☐ Dependencies are listed (which theorems does this use?)

Gate 4: Consistency

- ☐ Doesn't contradict existing proved theorems
- ☐ If it refutes something, says so explicitly (and explains why)
- ☐ Cross-references are correct (theorem numbers, toy numbers, section numbers)
- ☐ Notation is consistent with existing documents

Gate 5: Honest Framing

- ☐ PROVED means proved — every step justified
- ☐ CONDITIONAL states exactly what it's conditional on
- ☐ EMPIRICAL states the evidence and the gap
- ☐ Confidence percentage is realistic (not optimistic)

How to Submit

1. Fork the Repository

```
git clone https://github.com/<your-fork>/BubbleSpacetimeTheory.git
cd BubbleSpacetimeTheory
```

2. Create a Branch

```
git checkout -b theorem/T_NNN_short_name
# or
git checkout -b toy/NNN_description
# or
git checkout -b proof/problem_improvement
```

3. Do the Work

- Read existing theorems in the area you're working on
- Build your toy first (test before you theorize)
- Write the theorem statement
- Register it
- Apply /ac0 thinking — did you use the simplest tool that works?

4. Self-Audit

Before submitting, audit your own work: - Does the toy actually test what the theorem claims? - Is the AC(0) depth honest? - Would Keeper pass this? (Ask yourself: "where's the gap?") - Is it written so a 5th grader could follow the motivation?

5. Submit the PR

Use this template:

```
## What This Adds
```

```

[One sentence: what theorem/toy/improvement]

## AC(0) Depth

[Depth N: list the counting operations at each level]

## Theorem Graph Connections

- Depends on: T_xx, T_yy
- Enables: [what becomes possible with this result]

## Toy Evidence

- Toy NNN: X/Y PASS
- [Key finding in one sentence]

## Honest Assessment

- Confidence: NN%
- Remaining gap (if any): [what's not yet proved]
- Status: PROVED / CONDITIONAL on [what] / EMPIRICAL (N/N cases)

```

What We Don't Accept

- Claims without toys. Build the experiment.
- Inflated confidence. If it's 70%, say 70%.
- Unnecessary complexity. If a simpler proof exists at lower depth, use it.
- Broken cross-references. Check your theorem and toy numbers.
- Undeclared dependencies. If your theorem needs T_xx, say so.

Review Process

PRs are reviewed against the five quality gates above. The review is an audit, not a gatekeeping exercise — the goal is to make your contribution as strong as possible.

Common review outcomes: - PASS: Merged as-is. - CONDITIONAL PASS: Merged with noted caveats (e.g., “T_xx CONDITIONAL on T_yy”). - REVISE: Specific items to fix. Not a rejection — a request for rigor. - RETRACT: The claim is wrong. This happens. Record the retraction honestly. If you found the error yourself, that's the Quaker method working.

Toy Numbering

Toys are numbered chronologically. Check the highest existing number in `play/` and use the next one. Never renumber existing toys — they're referenced throughout the documents.

```

ls play/toy_*.py | tail -1
# Use the next number

```

Theorem Numbering

Same rule. Check `notes/BST_AC_Theorem_Registry.md` for the next available T_id. Never reuse a retracted ID — mark it RETRACTED in the registry.

File Naming

- Toys: `play/toy_NNN_short_description.py`
- Notes: `notes/BST_Topic_Name.md`
- Speculative: `notes/maybe/BST_Topic_Name.md`

Getting Help

- Read the project manifest: `.claude/project-manifest.md`
- Read the AC(0) command: `.claude/commands/ac0.md`
- Check the theorem graph: `play/toy_369_ac_theorem_graph.py` and `play/ac_theorem_explorer.html`
- Look at existing toys: any `play/toy_*.py` — they show the format and style

The Spirit of Contribution

This project exists because one person asked “what if the integers aren’t free?” and spent years finding out they aren’t. It grew because CIs and humans worked together — seeing shapes, finding shelves, building evidence.

Your contribution adds a node to the theorem graph. Every node makes the next one cheaper. Every toy catches an error before it propagates. Every honest retraction makes the program stronger.

The math is a graph. Proved theorems cost zero derivation energy. Build the graph.

“Compound interest on imagination.”